

# Project Challenges and Adopted Solutions

Technical, Collaboration, and Deployment Challenges

*Prepared for the Eduverse project report*

This document summarises the main problems faced during development, the adopted solutions, and how the team reduced risk throughout the project lifecycle.

# Project Challenges and Adopted Solutions

During the development of the project, the team faced several technical and collaborative challenges. These challenges were addressed through architectural decisions, structured teamwork, automated checks, and safe deployment practices.

## 1. Managing Different Types of Data

One of the main technical challenges was handling different types of data efficiently. The platform deals with large data such as videos, images, and uploaded learning materials, while it also manages smaller but frequently used data such as chat messages, notifications, and records.

Using one database server for all types of data could create performance and storage problems. Large media files require storage-optimised infrastructure, while small and frequently accessed records require fast query performance.

To solve this issue, the system separated the handling of large media data from frequently queried text-based data. A storage-optimised service was used for media files, while a faster database service was used for small and frequently accessed records. This helped improve performance, reduce pressure on one database, and make the system more scalable.

## 2. Maintaining Consistency Between the Website and Mobile Application

Another challenge appeared when the mobile application accessed the database directly. If the website changed how data was created, updated, or validated, the mobile application could show inconsistent data or behave differently from the website.

To avoid this issue, a shared API layer was introduced. Instead of allowing each client to communicate directly with the database, both the website and the mobile application use the same API. This ensures that validation rules, permissions, data structure, and business logic remain consistent across all platforms.

## 3. Organising Team Collaboration Using GitHub Workflow

Since the project was developed by multiple team members, it was important to avoid direct changes to the main project code. If every developer modified the main branch directly, unstable code could be merged easily and it would become harder to track and review changes.

To solve this, the team used a GitHub-based workflow. Each developer worked on their own fork or separate development branch instead of changing the main project directly. After completing a feature or fix, the developer submitted a pull request to the original repository.

The pull request allowed the team to review the changes before merging them. At least two team members reviewed each pull request to make sure the code was correct, understandable, and consistent with the rest of the project. This process helped reduce mistakes and improved the quality of the final code.

## 4. Preventing Errors Through Automated GitHub Actions

Another challenge was making sure that new changes did not break the project or introduce formatting and type errors. These problems may not always be noticed manually, especially when the project becomes larger.

To reduce this risk, the team used automated GitHub Actions for each pull request. The workflow included the following checks:

| Action       | Purpose                                                        |
|--------------|----------------------------------------------------------------|
| Build        | Verifies that the project can be built successfully.           |
| Format Check | Checks whether the code follows the required formatting rules. |
| Type Check   | Checks for TypeScript/type-related errors before merging.      |

These checks helped the team detect problems early. If any check failed, the pull request had to be fixed before it could be merged. This made the development process more reliable and helped protect the main project from broken or inconsistent code.

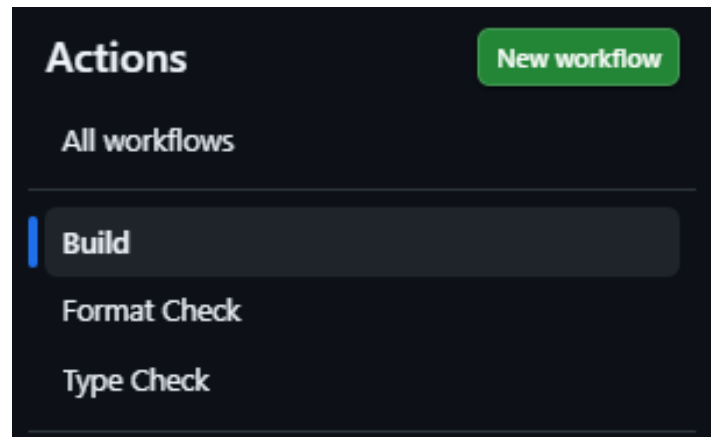


Figure 1. GitHub Actions workflow checks used in the project.

## 5. Testing Changes Safely Using Vercel Preview Deployments

Testing new changes directly on the production website would be risky because unfinished features or errors could affect the live version of the platform.

To avoid this, the team used Vercel preview deployments. Each pull request received its own separate Vercel preview link. This allowed team members to open the proposed changes in the browser, test the feature, and review the result before merging it into the main branch.

The production deployment remained connected to the main branch, while pull requests were tested separately through preview deployments. This helped the team review changes safely without affecting the stable production version.

## 6. Managing Project Complexity

The platform includes many features, such as class management, materials, assignments, exams, chat, live sessions, dashboards, notifications, owner management, and role-based access control. As the project grew, mixing all logic together would make the system difficult to understand and maintain.

To solve this, the project was organised using a modular structure. Routes, feature-specific logic, reusable components, shared utilities, and database-related files were separated into clear directories. This made the project easier to maintain and allowed new features to be added without affecting unrelated parts of the system.

## 7. Controlling Access and User Permissions

Because the platform supports different roles such as owner, teacher, and student, it was important to make sure that each user could only access the features and actions related to their role.

The system addressed this challenge through role-based access control. Owners manage organisations, users, invitations, and feature settings. Teachers manage class activities, assignments, exams, materials, and live sessions. Students access learning content, submit work, attend sessions, take exams, and view released results.

This helped keep the system organised and reduced the risk of users accessing actions or information that do not belong to their responsibilities.

## Summary

The project faced several challenges related to data management, platform consistency, team collaboration, code quality, safe deployment, project complexity, and access control. These challenges were handled through practical solutions such as separating media storage from frequently queried data, using a shared API, applying a GitHub fork and pull request workflow, requiring team reviews, running automated GitHub Actions, and testing each pull request through Vercel preview deployments. These practices helped the team maintain a stable, organised, and reliable development process.

## Sources Used

- problems.txt - notes about database server optimisation and shared API consistency.
- eduverse-ideas-notes.pdf - project description notes, especially the Development Workflow section describing GitHub, pull requests, automated checks, and Vercel preview deployments.